PERTEMUAN 10 · Praktik terbimbing

Bottom-Up Parsing: Shift-Reduce dan Operator Precedence

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 04 — Mengimplementasikan parser sederhana untuk validasi kode.

Sub-CPMK yang dinilai

Sub-CPMK 4.2: Membuat tabel aksi dan *goto* untuk parser SLR/LALR, lalu menguji dan men-*debug* konflik *shift-reduce* atau *reduce-reduce* yang muncul.

Cakupan dua pertemuan

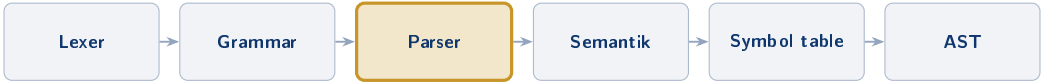
Pertemuan 10 menyiapkan model *shift-reduce* dan tabel operator precedence. Tabel ACTION/GOTO SLR/LALR diselesaikan pada Pertemuan 11; keduanya menjadi artefak Sub-CPMK 4.2.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
30	Ceramah: model *shift-reduce*, stack, dan handle
30	Latihan: menelusuri stack dan menandai handle
40	Lokakarya: tabel operator precedence
30	Praktik: kalkulator *shift-reduce* dalam C
20	Diskusi konflik dan jembatan ke LR

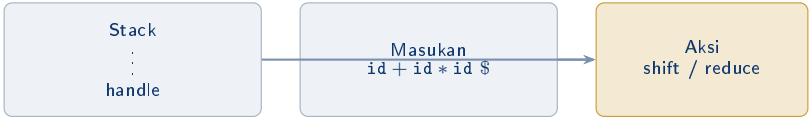
Porsi praktik dan latihan 100 menit. Keluaran hari ini menjadi data pendukung tabel ACTION/GOTO pada Pertemuan 11.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini memperdalam simpul **parser** dengan arah bottom-up: dari daun menuju akar.
- Model *shift-reduce* dan tabel precedence menjadi fondasi tabel ACTION/GOTO SLR/LALR.
- Konflik yang ditemukan hari ini didokumentasikan sebagai bahan *debug* pada Pertemuan 11.

Model Shift-Reduce



- Parser membaca token kiri ke kanan dan memutuskan *shift* atau *reduce*.
- *Shift*: dorong token ke stack.
- *Reduce*: ganti ruas kanan aturan dengan nonterminal.
- Terima ketika stack berisi simbol awal dan masukan habis.

Tata Bahasa Contoh dan Handle

Tata bahasa ekspresi

$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to \texttt{id}.$

- Tata bahasa ini *left-recursive*, sehingga cocok untuk bottom-up tetapi tidak cocok untuk top-down tanpa transformasi.
- **Handle**: substring yang cocok dengan ruas kanan aturan dan dapat direduksi menjadi nonterminal kiri.
- Pada $\texttt{id} + \texttt{id} * \texttt{id}$, handle pertama adalah $\texttt{id}$ paling kiri.
- $\texttt{id}$ direduksi menjadi F, lalu T, lalu E.

Penelusuran Shift-Reduce

Stack	Masukan	Aksi
	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	shift
$\texttt{id}$	$+\texttt{id} * \texttt{id}\ \$$	reduce $F \to \texttt{id}$
F	$+\texttt{id} * \texttt{id}\ \$$	reduce $T \to F$
T	$+\texttt{id} * \texttt{id}\ \$$	reduce $E \to T$
E	$+\texttt{id} * \texttt{id}\ \$$	shift $+$
$E+$	$\texttt{id} * \texttt{id}\ \$$	shift $\texttt{id}$
$E + \texttt{id}$	$*\texttt{id}\ \$$	reduce $F \to \texttt{id}$
$E + F$	$*\texttt{id}\ \$$	reduce $T \to F$
$E + T$	$*\texttt{id}\ \$$	shift $*$
...	reduce $F \to \texttt{id}$, $T \to T * F$, $E \to E + T$, lalu terima	
E	$\$$	terima

Keputusan diambil dari isi stack dan token saat ini, bukan dari seluruh masukan.

Keuntungan dan Keterbatasan Bottom-Up

Keuntungan

- menangani *left recursion* secara alami;
- mendukung grammar yang lebih luas;
- menjadi dasar parser LR;
- cocok untuk parser generator.

Keterbatasan

- tabel dan algoritma lebih kompleks;
- pesan kesalahan lebih sulit dibuat kontekstual;
- memerlukan konstruksi state yang sistematis.

Top-down dan bottom-up bukan pengganti satu sama lain; pilihan bergantung pada grammar dan kebutuhan diagnostik.

Viable Prefix

Definisi

Viable prefix adalah prefiks dari bentuk kalimat yang dapat muncul pada stack parser *shift-reduce* tanpa melewati handle.

- Menentukan apakah stack masih dapat diperluas menjadi bentuk kalimat valid.
- Jika stack bukan *viable prefix*, parser tidak mungkin mencapai simbol awal.
- Viable prefix menjadi dasar pembentukan state LR(0) pada Pertemuan 11.

Konflik Shift-Reduce

Definisi

Konflik *shift-reduce* terjadi ketika parser dapat memilih antara shift dan reduce untuk stack dan token yang sama.

Contoh

Pada $E \to E + E \mid E * E \mid \texttt{id}$, setelah stack berisi $E + E$ dan token berikutnya $*$, parser dapat:

- reduce $E + E$ menjadi E, atau
- shift $*$ untuk membentuk $E * E$.

Pilihan ini menentukan apakah penjumlahan atau perkalian dikelompokkan lebih dulu.

Konflik Reduce-Reduce dan Penyelesaiannya

Konflik *reduce-reduce*

Terjadi ketika dua aturan berbeda dapat diterapkan pada handle yang sama. Biasanya menandakan grammar terlalu ambigu atau aturan tumpang tindih.

- Selesaikan dengan mengubah grammar: precedence, associativity, atau factoring.
- Tambahkan aturan prioritas pada parser generator.
- Pilih shift atau reduce dengan kebijakan tetap yang didokumentasikan.
- Bison secara default memilih shift; keputusan ini dapat menyembunyikan bug jika tidak disengaja.

Operator Precedence Grammar

Syarat

1. tidak memiliki produksi ε;
2. tidak memiliki dua nonterminal bersebelahan pada ruas kanan.

- Parser dapat menentukan hubungan precedence antara terminal berdekatan tanpa tabel LR lengkap.
- Tiga relasi: $a \lessdot b$ (lebih rendah), $a \doteq b$ (sama), $a \gtrdot b$ (lebih tinggi).
- Simbol $<$, $=$, $>$ di sini adalah relasi precedence, bukan operator aritmetika.

Tabel Operator Precedence

	id	+	*	(	)
id		$\cdot>$	$\cdot>$		$\cdot>$
+	$<\cdot$	$\cdot>$	$<\cdot$	$<\cdot$	$\cdot>$
*	$<\cdot$	$\cdot>$	$\cdot>$	$<\cdot$	$\cdot>$
(	$<\cdot$	$<\cdot$	$<\cdot$	$<\cdot$	$\doteq$
)		$\cdot>$	$\cdot>$		$\cdot>$

Sel kosong menandakan kesalahan sintaksis. Tabel ini lebih kecil daripada tabel LR karena hanya memuat terminal, bukan himpunan item.

Contoh Relasi Precedence

Untuk $\texttt{id} + \texttt{id} * \texttt{id}$:

- $\texttt{id} \cdot > +$: id harus direduksi sebelum $+$.
- $+ < \cdot \texttt{id}$: $+$ menunggu operand.
- $+ < \cdot *$: $*$ memiliki precedence lebih tinggi.
- $* \cdot > \texttt{id}$: $*$ menunggu operand.

Cara membaca

Parser membandingkan terminal teratas stack dengan token saat ini. Relasi menentukan shift atau reduce: $< \cdot$ dan $\doteq$ mendorong shift; $\cdot >$ memicu reduce.

Algoritma Operator Precedence

1. Cari terminal teratas pada stack, sebut a.
2. Bandingkan a dengan token saat ini b.
3. Jika $a \lessdot b$, shift b.
4. Jika $a \doteq b$, shift atau tangani kurung.
5. Jika $a \gtrdot b$, reduce handle teratas.
6. Jika sel kosong, laporkan kesalahan.

Handle adalah substring stack yang cocok dengan ruas kanan aturan. Setelah reduce, bandingkan kembali terminal teratas dengan token saat ini.

Contoh Penelusuran Operator Precedence

Untuk $\texttt{id} + \texttt{id} * \texttt{id}$:

1. shift `id`, reduce ke F, lalu T, lalu E.
2. shift $+$.
3. shift `id`, reduce ke F, lalu T.
4. $+ < \cdot *$, shift $*$.
5. shift `id`, reduce ke F.
6. $* \cdot > \$$: reduce $T \to T * F$.
7. $+ \cdot > \$$: reduce $E \to E + T$.
8. terima.

Urutan ini memaksa perkalian direduksi sebelum penjumlahan.

Kalkulator *Shift-Reduce* dalam C

```
30 static void reduce_top(int *values, int *vtop,
31                        char *ops, int *otop) {
32     int right = values[(*vtop)--];
33     int left = values[(*vtop)--];
34     char op = ops[(*otop)--];
35     values[++(*vtop)] = apply(left, op, right);
36     printf("  reduce: %d %c %d -> %d\n",
37            left, op, right, values[*vtop]);
38 }
```

Listing 1: Fungsi `reduce_top` pada `shift_reduce.c`

- Pop dua nilai dan satu operator.
- Hitung hasil dengan `apply`.
- Dorong hasil kembali.
- Program mencetak urutan shift dan reduce.
- Masukan 2+3*4 menghasilkan 14.

Deteksi Kesalahan

Kapan kesalahan terdeteksi

Ketika relasi precedence tidak terdefinisi untuk pasangan terminal yang berdekatan.

Contoh

Pada `id` $+$ $+$`id`, setelah shift $+$ kedua, tabel tidak memiliki relasi antara $+$ dan $+$. Parser melaporkan kesalahan pada token kedua dan dapat melewati token sampai titik aman.

Operator precedence tidak menangani grammar dengan produksi ε atau dua nonterminal bersebelahan; gunakan parser LR untuk grammar yang lebih umum.

Perbandingan Top-Down dan Bottom-Up

Aspek	Top-down	Bottom-up
Arah	Akar ke daun	Daun ke akar
Left recursion	Harus dihapus	Dapat ditangani
Grammar	LL(1) dan variannya	LR, SLR, LALR, precedence
Tabel	Satu entri per nonterminal	Satu entri per state
Pesan kesalahan	Mudah kontekstual	Lebih sulit
Implementasi	Recursive descent	Stack dan tabel
Contoh	Parser LL(1)	Bison, Yacc, operator precedence

Operator precedence adalah bentuk sederhana bottom-up; LR pada Pertemuan 11 lebih umum.

Kegiatan Mahasiswa di Kelas

Lokakarya: penelusuran dan kalkulator

1. Telusuri shift-reduce untuk `id` $*$ `id` $+$ `id`.
2. Tandai handle pada setiap langkah reduce.
3. Bangun tabel operator precedence untuk $+$, $-$, $*$, $/$, dan kurung.
4. Modifikasi `shift_reduce.c` agar mendukung $-$, $/$, dan kurung.
5. Uji dengan ekspresi yang mengandung precedence dan associativity berbeda.

Minimal 10 ekspresi uji. Data penelusuran dan tabel precedence disimpan untuk Pertemuan 11.

Tugas dan Artefak

Artefak Sub-CPMK 4.2 (bagian pertama)

Tabel operator precedence, penelusuran *shift-reduce*, kalkulator C, dan catatan konflik *shift-reduce* atau *reduce-reduce* yang ditemukan.

- Penelusuran manual benar dan setiap handle ditandai.
- Tabel precedence lengkap untuk empat operator dan kurung.
- Kalkulator mendukung $+$, $-$, $*$, $/$, dan kurung.
- Minimal 10 ekspresi uji dengan hasil yang benar.
- Konflik yang muncul dicatat untuk *debug* tabel ACTION/GOTO pada Pertemuan 11.

Artefak lengkap Sub-CPMK 4.2 (tabel aksi dan *goto* SLR/LALR) diselesaikan pada Pertemuan 11 dan dilaporkan pada Praktikum 3.

Rangkuman dan Jembatan

- Parser bottom-up membangun pohon dari daun ke akar dengan *shift* dan *reduce*.
- Handle adalah substring yang dapat direduksi; viable prefix menentukan stack yang masih valid.
- Konflik *shift-reduce* dan *reduce-reduce* diselesaikan dengan mengubah grammar atau kebijakan tetap.
- Operator precedence memakai relasi $<\cdot$, $\doteq$, $\cdot>$ dan tabel yang lebih kecil daripada tabel LR.
- Data penelusuran dan tabel precedence menjadi fondasi tabel ACTION/GOTO SLR/LALR.

Pertemuan berikutnya

Pertemuan 11 membangun item LR(0), tabel ACTION/GOTO SLR, canonical LR, dan LALR, lalu men-*debug* konflik yang muncul.